

## PROJECT 2: AI-ENHANCED SMART LIGHTING SYSTEM

In this project, you'll create an AI-enhanced smart lighting system that adjusts the lighting in your home based on your daily routines, natural light levels, and personal preferences.

### Step 1: Set Up Your Smart Lights

- 1. Choose Smart Lights:** Select smart lights compatible with your smart home hub. Popular options include Philips Hue, LIFX, and TP-Link Kasa.
- 2. Install the Lights:** Replace your existing light bulbs with smart lights in the rooms where you want to automate lighting.
- 3. Connect Lights to Your Hub:** Use the smart light app to connect each light to your smart home hub (e.g., Amazon Echo, Google Nest Hub).

### Step 2: Configure Basic Lighting Settings

- 1. Set Up Default Settings:** In the smart light app, configure default settings for brightness and color temperature based on the room's function (e.g., bright white for the kitchen, warm white for the living room).
- 2. Create Basic Schedules:** Set up basic schedules for turning lights on and off at specific times of the day.

### Step 3: Integrate AI for Dynamic Lighting Adjustment

- 1. Choose an AI Platform:** Select an AI platform to analyze data and adjust lighting dynamically. Google Cloud AI and AWS AI Services are good options.
- 2. Set Up the AI Platform:** Create an account and set up your chosen AI platform. Obtain the necessary API keys.
- 3. Develop AI Algorithms:** Create AI algorithms to analyze your routines, natural light levels, and preferences to adjust lighting dynamically. Below is a simplified example using Python and a machine learning library:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Sample lighting data
data = pd.DataFrame({
    'time_of_day': [6, 12, 18, 22],
    'natural_light': [0.1, 0.8, 0.4, 0.1],
    'preferred_brightness': [0.7, 0.5, 0.6, 0.3]
})

# Train a model
X = data[['time_of_day', 'natural_light']]
y = data['preferred_brightness']
model = LinearRegression().fit(X, y)

# Predict preferred brightness
def predict_brightness(time_of_day, natural_light):
    return model.predict([[time_of_day, natural_light]])[0]

# Example usage
current_brightness = predict_brightness(14, 0.5)
print(f"Predicted brightness level: {current_brightness}")
```

### Step 4: Automate Lighting Adjustments

- 1. Set Up Automation Rules:** Use your smart home hub's app to set up automation rules based on the AI recommendations. For example, if the AI suggests increasing brightness in the afternoon when natural light is low, configure the lights to adjust accordingly.
- 2. Test Your System:** Test the setup by observing how the lighting adjusts throughout the day and checking if it aligns with your preferences.

custom commands for various smart home functions, from controlling appliances to managing security systems.

### Example: AI-Driven Predictive Maintenance

- 1. Monitor Device Health:** Use AI to monitor the health and performance of your smart home devices. This can involve tracking metrics such as energy consumption, operating temperature, and usage patterns.

- 2. Predict Maintenance Needs:** Develop AI models to predict when devices may require maintenance or replacement based on the monitored data.
- 3. Automate Alerts and Actions:** Set up automations to alert you when maintenance is needed and, if possible, take preemptive actions. For example, if the AI detects that a filter in your HVAC system needs replacement, it can notify you and order a new filter automatically. 